End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 – Software Production Engineering Mini-Project Report

Abhijit Dibbidi (IMT2023054) Abhijit.Dibbidi@iiitb.ac.in

GitHub repository: `https://github.com/erakin027/calculator-devops`
Docker Hub image: `https://hub.docker.com/r/erakin027/calculator-devops`
Stack: Java 17, Spring Boot 3, Maven, JUnit 5, Git/GitHub, Jenkins, Docker, Docker Compose, Nginx

1 Overview

The project is a small Calculator REST service (add, subtract, multiply, divide, plus an Actuator health probe) wrapped in a fully automated pipeline. A push to GitHub triggers Jenkins through a webhook; Jenkins builds, tests, containerizes, publishes the image to Docker Hub and redeploys the Compose stack, then emails the result. No manual step is needed between `git push` and a running, health-checked deployment.

2 Architecture

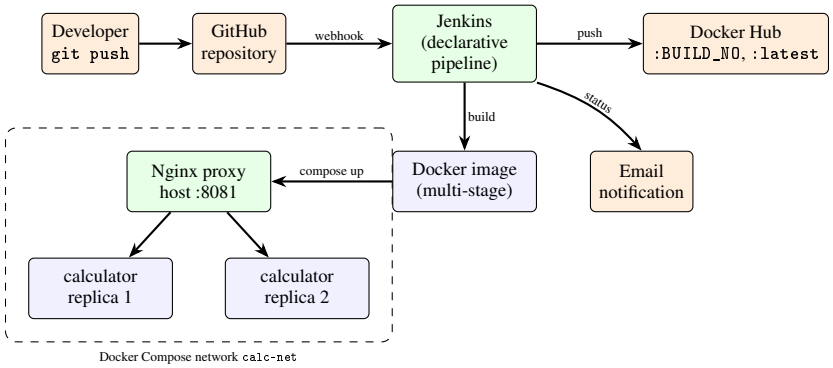


Figure 1: CI/CD architecture: GitHub → Jenkins → Docker Hub → Docker Compose (Nginx + two API replicas).

3 Application

The API exposes `GET /api/v1/{add,subtract,multiply,divide}?a=&b=` returning JSON, for example

```
{"operation":"add","a":2.0,"b":3.0,"result":5.0}
```

A `@RestControllerAdvice` converts divide-by-zero, missing and non-numeric parameters into HTTP 400 with a structured error body. `/actuator/health` is the container health probe. There are 13 tests: 5 JUnit 5 unit tests for the service (including divide by zero) and 8 MockMvc integration tests covering every endpoint, the error cases and the health endpoint; Maven Surefire writes the JUnit XML reports that Jenkins publishes.

4 Pipeline stages

Stage	Description
Checkout	`checkout scm` pulls the commit that fired the webhook.
Build	`mvn clean package -DskipTests` compiles and produces the executable JAR.
Test	`mvn test`; `junit` publishes the XML results and any failing test fails the build.
Docker Build	Builds the multi-stage image tagged with the Jenkins build number and `latest`.
Push	Logs in with the Jenkins credential `dockerhub-creds` (password via `--password-stdin`, never hard-coded) and pushes both tags.
Deploy	`docker compose up -d --remove-orphans` using the new tag, then polls `/actuator/health` through the proxy and fails the build if the stack is not UP.
Post	Success or failure email to the student with the build URL.

5 Containerization and deployment

Dockerfile. Stage 1 (`maven:3.9-eclipse-temurin-17`) copies `pom.xml` first and runs `dependency:go-offline` so dependencies form a cached layer, then builds the JAR. Stage 2 (`eclipse-temurin:17-jre-alpine`) copies only the JAR, runs as a non-root user `app`, and declares `EXPOSE 8080`. A `.dockerignore` keeps `target/`, `.git` and IDE files out of the build context.

Compose. `docker-compose.yml` defines a bridge network, two `calculator` replicas with a `wget`-based health check on `/actuator/health`, `restart: unless-stopped`, environment variables, and an Nginx reverse proxy (published on host port 8081) that starts only after the replicas are healthy. Because the project name and tags are fixed, re-running the

pipeline simply recreates changed containers, so it is idempotent.

6 Challenges faced

- **Jenkins could not run Docker:** the `jenkins` user lacked permission on the Docker socket; fixed by adding it to the `docker` group and restarting Jenkins.
- **Webhook could not reach a local Jenkins:** GitHub cannot call `localhost`; solved with an HTTPS tunnel (ngrok) and by setting the Jenkins URL accordingly (the URL changes on restart).
- **Jenkins tool names did not resolve:** the Jenkinsfile refers to tools by name (`Maven3`, `JDK17`) so that it stays portable and holds no machine paths, but the first run failed because those names did not exist on my Jenkins; defining them under *Manage Jenkins* → *Tools* fixed it.
- **Email delivery:** Gmail rejects normal passwords for SMTP; an app password with SSL on port 465 was required.

7 Evidence

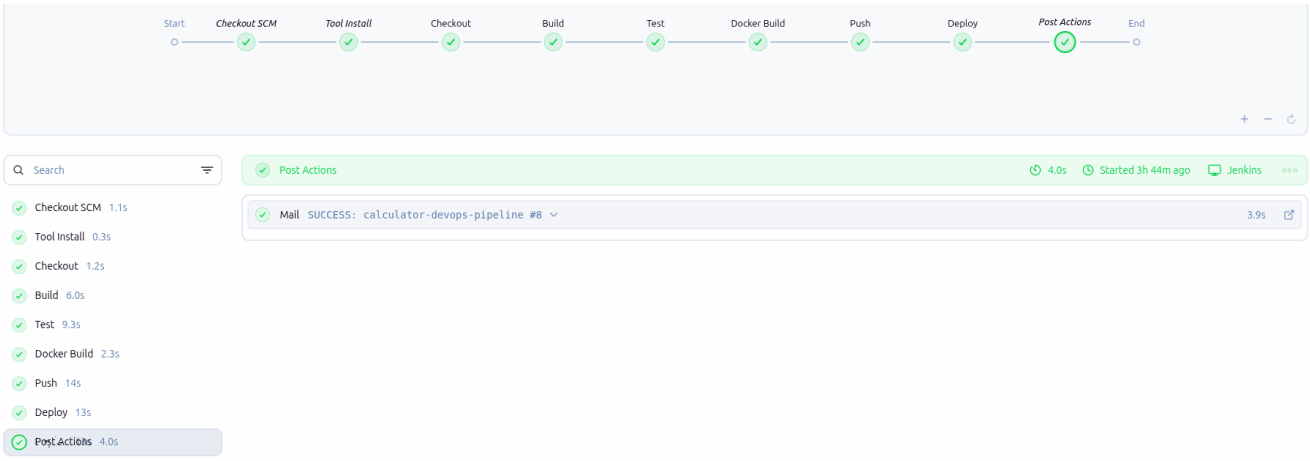


Fig. 2: Green Jenkins pipeline run (build #8): all stages Checkout, Build, Test, Docker Build, Push, Deploy and Post Actions passed

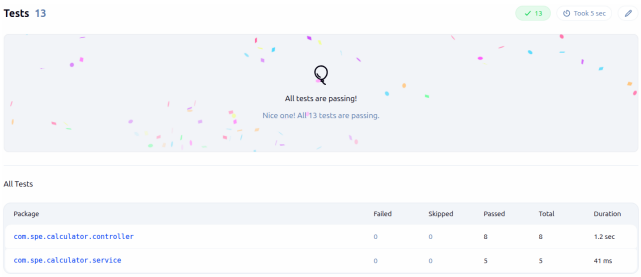


Fig. 3: JUnit results published in Jenkins (13 passed)



Fig. 4: Build started automatically by a GitHub push

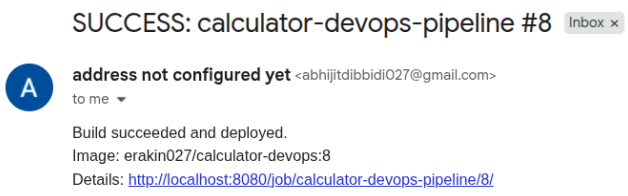


Fig. 5: Build-status email notification (success)

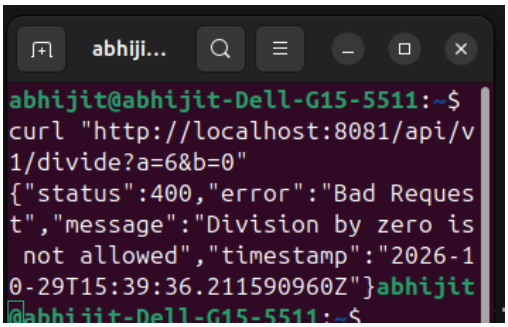


Fig. 6: Sample API response: divide by zero returns HTTP 400 with a JSON error body

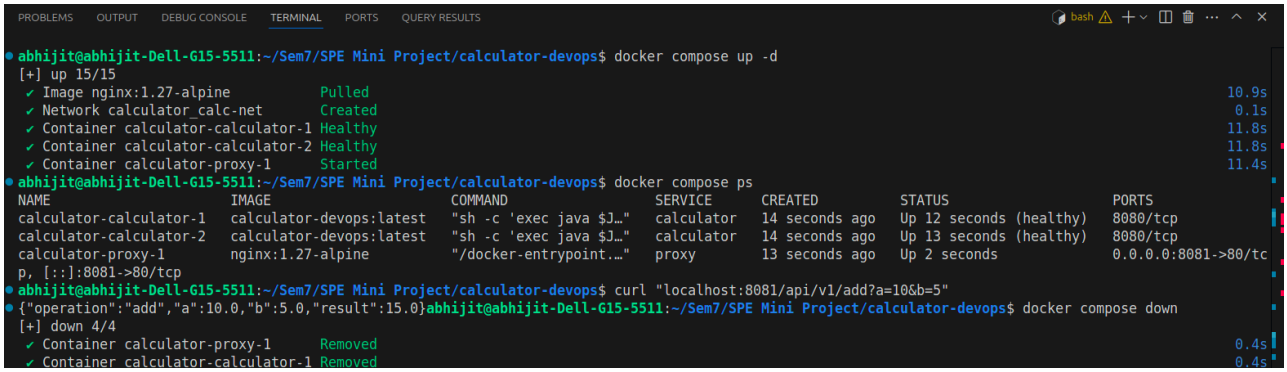


Fig. 7: Compose stack running: two calculator replicas and the Nginx proxy, all healthy, with a sample API call

Further evidence (Jenkins build history, webhook polling log, merged pull requests, local test run, non-root container check) is in the repository under `docs/screenshots/`.